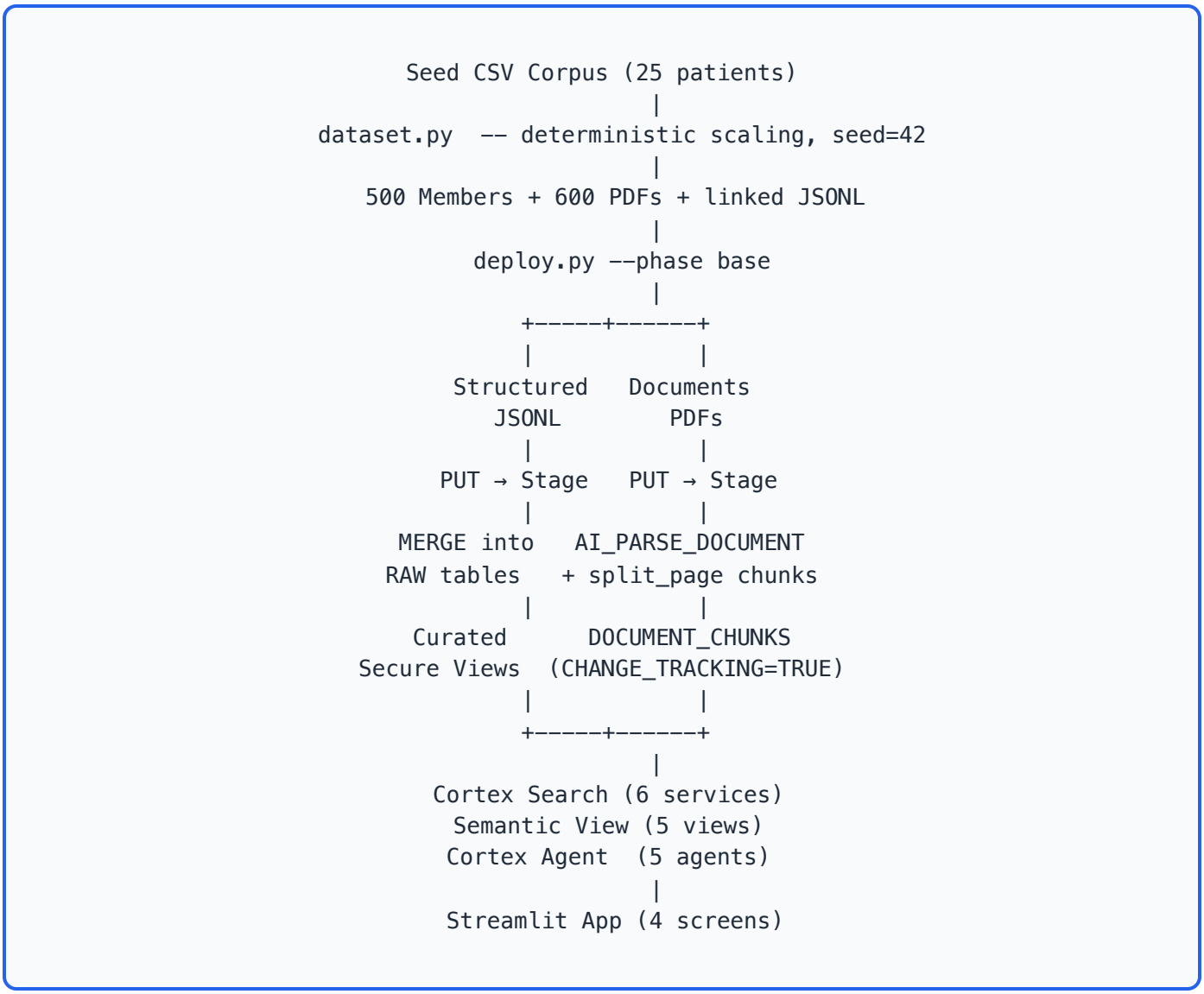
# CareLens360: Complete Workflow Reference

Evidence-First Patient and Member Copilot — Snowflake Hackathon 2026

**Database:** `CARELENS360_MVP` | **Account:** `WTOJJON-EYC21064` | **Warehouse:** `COMPUTE_WH`

## 1. Architecture Overview




**Schema Layout**

| Schema | Purpose | Key Objects |
|---|---|---|
| `RAW` | Staged ingestion, manifests, audit | 16 entity tables, DATA_STAGE, PIPELINE_AUDIT, DOCUMENT_PARSE_RESULTS, WORKFLOW_CONFIG |
| `CURATED` | Business logic, dimensions, facts | 13 secure views (DIM_MEMBER, FACT_ENCOUNTER, MEMBER_RISK_FACTORS, MEMBER_360_SNAPSHOT, etc.) + |

| | | |
|---|---|---|
| | | DOCUMENT_CHUNKS table |
| `SEMANTIC` | AI services layer | 5 semantic views, 6 Cortex Search Services, plan-scoped secure views |
| `APP` | Governed application layer | 15 governed views, ACCESS_AUDIT, USER_ENTITLEMENTS, 5 Agents, Streamlit app, REFRESH_TASK |

## 2. Data Generation

### Module: `src/carelens/dataset.py` (388 lines)

Deterministically scales a 25-patient CSV seed corpus to the full MVP volume using seed 42 and as-of date 2026-09-30.

| Function | Purpose |
|---|---|
| `digest(path)` | SHA-256 checksum of a file for version integrity |
| `read_seed(root)` | Reads and validates the 25-patient CSV seed corpus |
| `pdf(path, pages)` | Generates multi-page PDFs using ReportLab |
| `validate(tables)` | Checks row counts, unique keys, referential integrity |
| `generate(seed_root, output, config)` | Main entry: produces all JSONL + 600 PDFs + oracle + evaluation cases |

### Generated Volumes

| Entity | Count | Entity | Count |
|---|---|---|---|
| Members | 500 | Medications | 1,000 |
| Encounters | 5,000 | Clinical Notes | 500 (PDFs) |
| Medical Claims | 10,000 | Safety/Regulatory Docs | 100 (PDFs) |
| Lab Results | 3,000 | Conditions | 1,080 |
| Pharmacy Fills | 4,630 | Risk Factors (oracle) | 3,000 |
| Adverse Events | 160 | Care Gaps | 2,320 |

### Conditions Covered

Diabetes (T2D), Hypertension (HTN), Chronic Kidney Disease (CKD3), Asthma, Heart Failure (HF)

### CLI Command

```
carelens generate --seed-folder /path/to/hcls360_dataset --output data/generated/v5
```

## 3. Risk Stratification

**Module: `src/carelens/risk.py` (92 lines)**

Transparent, rule-based scoring. Each factor contributes at most once per member per run.

| Factor | Points | Trigger Rule |
|---|---|---|
| Uncontrolled Lab | 30 | Latest observation per test within 180 days has source HIGH/LOW/ABNORMAL flag |
| Recent ED | 20 | At least one distinct ED encounter within 90 days |
| Adherence Concern | 15 | Active non-PRN medication PDC < 80% over 183 days |
| Multiple Chronic Conditions | 15 | At least 2 distinct active condition keys |
| Care Gap | 10 | Unresolved TRC follow-up gap due on or before as-of date |
| Recent Adverse Event | 10 | Recorded event within 90 days |

| Score Range | Tier | Score Range | Tier |
|---|---|---|---|
| 0 – 24 | Low | 50 – 74 | High |
| 25 – 49 | Moderate | 75 – 100 | Critical |

**Key Functions**

| Function | Purpose |
|---|---|
| `tier(score)` | Maps 0-100 integer to Low/Moderate/High/Critical |
| `covered_days(fills, as_of, window)` | Computes deduplicated calendar-day PDC from pharmacy fills |
| `evaluate(member, tables, config)` | Evaluates all 6 factors with MET/NOT_MET/UNKNOWN states and source IDs |

## 4. SQL Generation

**Module: `src/carelens/sql.py` (335 lines)**

Generates all DDL and DML for the four-schema layout. No SQL is hardcoded in files; everything is built programmatically.

| Function | Generates |
|---|---|
| `identifier(value)` | Validates Snowflake identifiers; rejects SQL injection |
| `literal(value)` | Escapes string literals for SQL |
| `foundation(database)` | Database, 4 schemas, stages, file formats, system tables, 13 entity tables |
| `load_sql(database, entity)` | 3-statement COPY/MERGE pipeline per entity from staged JSONL |
| `curated(database, config)` | 13 secure views: dimensions, facts, risk factors with lineage, MEMBER_360_SNAPSHOT |
| `governed(database)` | Row-access policy, APP views with entitlement filtering, WORKFLOW_STATE, DOCUMENT_STATUS |
| `semantics(database, suffix, plan_id)` | CREATE SEMANTIC VIEW with 11 tables, relationships, 10 metrics, synonyms |

**Curated Layer Objects**

| View | Type | View | Type |
|---|---|---|---|
| DIM_MEMBER | Dimension | FACT_CLAIM | Fact |
| DIM_PROVIDER | Dimension | CARE_GAPS | Fact |
| DIM_CONDITION | Dimension | ADVERSE_EVENTS | Fact |
| DIM_MEDICATION | Dimension | MEMBER_RISK_FACTORS | Derived |
| FACT_DIAGNOSIS | Fact | MEMBER_360_SNAPSHOT | Derived |
| FACT_ENCOUNTER | Fact | DOCUMENT_CHUNKS | Table |
| FACT_LAB_RESULT | Fact | | |
| FACT_MEDICATION | Fact | | |

## 5. Document Processing Pipeline

**Module: `src/carelens/chunks.py` (50 lines)**

| Function | Purpose |
|---|---|
| `split_page(text, count_tokens, max=800, overlap=75)` | Binary-search chunking: finds max word span within token limit, then walks backward for measured overlap. Returns chunks with text, token count, character offsets, and exception tags (SHORT_PAGE, RESIDUAL). |

**Module: `snowflake/refresh.py` (115 lines)**

Snowpark stored procedure handler that runs inside Snowflake as `REFRESH_WORKFLOW`.

| Function | Purpose |
|---|---|
| `run_wrapper(session, parse_limit)` | Procedure entry point; delegates to `run()` |
| `run(session, database, parse_limit)` | Incremental pipeline: refreshes stage, finds unparsed docs, calls AI_PARSE_DOCUMENT with LAYOUT mode + page_split, measures tokens with COUNT_TOKENS, creates chunks, commits transactionally per document, audits to PIPELINE_AUDIT |

**Pending selection:** a manifest document (ID + SHA256) is pending unless it has an error-free row in `RAW.DOCUMENT_PARSE_RESULTS` *and* chunks in `CURATED.DOCUMENT_CHUNKS` for the same SHA (CTE + LEFT JOIN, ordered by ID, bounded by `parse_limit`). Successful parses store `ERROR` as SQL NULL (bound via `NULLIF(?, '')`, since Snowpark binds Python `None` as the string 'None'). Entity reloads are not done inside the procedure (temp tables are unsupported there); the base phase loads structured data.

### Processing Flow

```
1. PDF uploaded to @RAW.DATA_STAGE
2. Registered in RAW.DOCUMENT_MANIFEST (ID, SHA256, metadata)
3. AI_PARSE_DOCUMENT(TO_FILE('@DATA_STAGE', path), {mode:'LAYOUT', page_split:TRUE}, TRUE)
4. Parse result stored in RAW.DOCUMENT_PARSE_RESULTS
5. Each page split into chunks (500-900 tokens, 50-100 overlap)
6. Token counting via SNOWFLAKE.CORTEX.COUNT_TOKENS('snowflake-arctic-embed-m-v1.5')
7. Chunks inserted into CURATED.DOCUMENT_CHUNKS (transactional per document)
8. CHANGE_TRACKING on DOCUMENT_CHUNKS feeds Cortex Search incremental refresh
```

### Chunk Metadata Preserved

`CHUNK_ID, DOCUMENT_ID, MEMBER_ID, PATIENT_ID, PLAN_ID, DOCUMENT_TYPE, DOCUMENT_DATE, SOURCE_SYSTEM, AUTHOR_ROLE, CONDITION_GROUP, MEDICATION_NAME, PAGE_NUMBER, SECTION, SOURCE_FILE, CHUNK_TEXT, START_OFFSET, END_OFFSET, TOKEN_COUNT, OVERLAP_TOKENS, CHUNK_EXCEPTION, TOKENIZER, SHA256, CLASSIFICATION`

## 6. Cortex Search Configuration

Six search services created in `SEMANTIC` schema, all with 1-hour target lag and incremental refresh:

| Service | Scope | Source Filter |
|---|---|---|
| `SHARED_EVIDENCE_SEARCH` | Safety/regulatory docs (no member) | `PLAN_ID='SHARED'` |
| `CARE_DOCUMENT_SEARCH` | All member-specific docs | `PLAN_ID!='SHARED'` |
| `CARE_DOCUMENT_SEARCH_PLAN_1` | Plan 1 members only | `PLAN_ID='PLAN-1'` |
| `CARE_DOCUMENT_SEARCH_PLAN_2` | Plan 2 members only | `PLAN_ID='PLAN-2'` |
| `CARE_DOCUMENT_SEARCH_PLAN_3` | Plan 3 members only | `PLAN_ID='PLAN-3'` |
| `CARE_DOCUMENT_SEARCH_PLAN_4` | Plan 4 members only | `PLAN_ID='PLAN-4'` |

**Searchable:** `CHUNK_TEXT`
**Filterable Attributes:** `MEMBER_ID, PATIENT_ID, DOCUMENT_TYPE, DOCUMENT_DATE, SOURCE_SYSTEM, AUTHOR_ROLE, CONDITION_GROUP, MEDICATION_NAME`
**Returned to the agent per result:** `doc_id` (CHUNK_ID = `DOCUMENT_ID:SHA256:PAGE:INDEX`), `doc_title` (DOCUMENT_ID), `text`, a `columns` object with PAGE_NUMBER, SOURCE_FILE, DOCUMENT_DATE, DOCUMENT_TYPE and MEDICATION_NAME, plus `stage_path` / `relative_path` pointing at the source PDF on `@RAW.DATA_STAGE`. The extra columns appear only because the agent spec declares them (see section 8).

## 7. Semantic View Design

**Module:** `src/carelens/sql.py` → `semantics()`

Five semantic views: 1 main + 4 plan-scoped. Each has 11 tables, foreign-key relationships, and 10 metrics.

| Metric | Definition |
|---|---|
| member_count | `COUNT(DISTINCT MEMBER_ID)` |
| encounter_count | `COUNT(DISTINCT ENCOUNTER_ID)` |
| emergency_visit_count | `COUNT(DISTINCT IFF(ENCOUNTER_TYPE='ED', ENCOUNTER_ID, NULL))` |
| total_claim_amount | `SUM(BILLED_AMOUNT)` |
| average_a1c | `AVG(IFF(TEST='HbA1c', VALUE, NULL))` |
| abnormal_lab_count | `COUNT(DISTINCT IFF(FLAG IN ('HIGH','LOW','ABNORMAL'), LAB_ID, NULL))` |
| medication_adherence_rate | `AVG(IFF(NOT IS_PRN AND STATUS='active', PDC, NULL))` |
| care_gap_count | `COUNT(DISTINCT IFF(STATUS IN ('OPEN','MISSED'), GAP_ID, NULL))` |
| high_risk_member_count | `COUNT(DISTINCT IFF(RISK_TIER IN ('High','Critical'), MEMBER_ID, NULL))` |
| follow_up_gap_count | `COUNT(DISTINCT IFF(FOLLOW_UP_GAP_COUNT>0 AND RISK_TIER IN ('High','Critical'), MEMBER_ID, NULL))` |

## 8. Cortex Agent

### Module: `src/carelens/agent.py` (99 lines)

Generates YAML specification for `CREATE AGENT`. Five agents: 1 admin-only global + 4 plan-scoped. Orchestration budget is **300 seconds / 32,000 tokens** (a member summary at 120s timed out live); the orchestration instructions ask for the member snapshot and risk factors in one PatientAnalytics call to keep tool calls low.

| Tool | Type | Target |
|---|---|---|
| PatientAnalytics | `cortex_analyst_text_to_sql` | PATIENT_MEMBER_360_SEMANTIC_VIEW (plan-scoped) |
| ClinicalDocumentSearch | `cortex_search` | CARE_DOCUMENT_SEARCH (plan-scoped) |
| SharedEvidenceSearch | `cortex_search` | SHARED_EVIDENCE_SEARCH |

### Search tool resources: `search_resource(database, service)`

| Setting | Value | Purpose |
|---|---|---|
| `max_results` | 5 | Bounded retrieval per call |
| `id_column` / `title_column` | CHUNK_ID / DOCUMENT_ID | Citation identity, verified by the app against `APP.DOCUMENT_CHUNKS` |
| `stage_path` / `relative_path_column` | `@<DB>.RAW.DATA_STAGE` / SOURCE_FILE | Links each citation to the source PDF |
| `columns_and_descriptions` | CHUNK_TEXT (searchable), DOCUMENT_ID, PAGE_NUMBER, SOURCE_FILE, DOCUMENT_DATE (datetime), DOCUMENT_TYPE, PATIENT_ID, MEMBER_ID, CONDITION_GROUP, MEDICATION_NAME | Returns page/source/date with every result and documents filter usage ( `@eq` identity, `@contains` arrays) |

The response instruction also tells the agent that the third `:` segment of `doc_id` is the original page and that the file is `documents/<DOCUMENT_ID>.pdf` as a fallback (both conventions hold for all 1,100 live chunks).

**Redeploy note:** `CREATE OR REPLACE AGENT` drops existing grants; after an agent-only redeploy re-run `GRANT USAGE ON ALL AGENTS IN SCHEMA <DB>.APP TO ROLE CARE_ADMIN`.

### Response Contract (5 sections)

```
## Summary
## Key Evidence
## Risk Drivers
## Evidence Citations
## Limitations
```

**Module: `src/carelens/response.py` (103 lines)**

| Function | Purpose |
|---|---|
| `public(value)` | Strips hidden reasoning (thinking/reasoning/signature) from response payloads |
| `parse_response(content)` | Parses SSE/JSON from Cortex Agent into {text, tool_results, annotations}. Raises explicit errors for a truncated/malformed stream, a non-JSON response, and an agent that reached its time budget, instead of a raw JSON decode error |
| `validate_format(text)` | Enforces exact 5-heading order and trailing disclaimer |

## 9. Deployment Pipeline

**Module: `src/carelens/deploy.py` (225 lines)**

Six-phase deployment with JSONL evidence logging and managed-database guard.

| Phase | Method | What It Does |
|---|---|---|
| base | `base(data)` | Creates DB/schemas/tables, validates & uploads documents (checksum verified), loads JSONL via MERGE, creates curated + governed views |
| procedure | `procedure(repo)` | Bundles Python into ZIP, creates REFRESH_WORKFLOW stored procedure + suspended REFRESH_TASK |
| parse | `parse(limit)` | Calls REFRESH_WORKFLOW with bounded limit (1-600) for incremental document parsing |
| services | `search_and_agents()` | Creates 6 Search services, 5 semantic views, 5 agents with plan-scoped secure views |
| roles | `roles()` | Creates 5 RBAC roles with tiered schema/object grants |
| app | `app(repo)` | Uploads Streamlit files to CODE_STAGE, creates CARELENS360 app (CARE_ADMIN only) |

### CLI Commands

```
carelens deploy --warehouse COMPUTE_WH --phase base     --apply --confirm-database CARELENS360_
carelens deploy --warehouse COMPUTE_WH --phase procedure --apply --confirm-database CARELENS360
carelens deploy --warehouse COMPUTE_WH --phase parse --parse-limit 600 --apply --confirm-databa
carelens deploy --warehouse COMPUTE_WH --phase services  --apply --confirm-database CARELENS360
carelens deploy --warehouse COMPUTE_WH --phase roles     --apply --confirm-database CARELENS360
carelens deploy --warehouse COMPUTE_WH --phase app       --apply --confirm-database CARELENS360
```

## 10. Streamlit Application

### Module: `app/streamlit_app.py` (156 lines)

Four-screen evidence copilot deployed as a Snowflake-hosted Streamlit app. Calls `/api/v2/databases/<DB>/schemas/APP/agents/<plan agent>:run` via `_snowflake.send_snow_api_request` with `"stream": false` and a 360-second timeout; catches `SnowparkClientException`.

| Screen | Features |
|---|---|
| **Member Search** | Filter by risk tier and condition key; member count metric; data table + risk tier bar chart |
| **Member 360** | Demographics snapshot; 7 detail sections (diagnoses, medications, encounters, labs, claims, care gaps, risk factors); audit logged |
| **Evidence Copilot** | Natural-language question box; plan-scoped Cortex Agent invocation; citation verification against source chunks; conflict/insufficient evidence warning banners; "Evidence/tool inputs" expandable panel; scope refusal detection; conversation history (last 10 messages) |
| **Population Insights** | High+Critical member count; risk tier distribution chart; risk-by-condition pivot table; utilization/adherence/care-gap metrics table; audit logged |

### Safety Guards

- Pipeline status check — blocks if refresh is running/failed
- Document readiness check — blocks if required documents pending/failed
- Citation plan/member/chunk verification — withholds unverifiable answers
- Context invalidation on data version change
- Synthetic data disclaimer banner

## 11. Governance

### Five Roles

| Role | Access Level |
|---|---|
| `CARE_ADMIN` | Full: all schemas, raw write, agents, search, semantic views, task control, entitlement management, Streamlit |
| `CARE_DATA_ENGINEER` | Pipeline: RAW/CURATED/SEMANTIC read+write, stages, procedure, file format |
| `CARE_ANALYST` | Read: APP governed views only |
| `CARE_REVIEWER` | Read: APP governed views only |
| `CARE_DEMO_USER` | Streamlit only (gated until isolation tests pass) |

### Data Controls

- **Row Access Policy:** `MEMBER_PLAN_POLICY` filters by USER_ENTITLEMENTS (plan-based or `*` wildcard)
- **Classification:** `CLASSIFICATION='SYNTHETIC'` column on DIM_MEMBER, RISK_FACTORS, DOCUMENT_CHUNKS, USER_ENTITLEMENTS
- **Access Audit:** Every Streamlit operation logged to `APP.ACCESS_AUDIT` (request_id, user, plan, operation, status)
- **Pipeline Audit:** Every parse/refresh run logged to `RAW.PIPELINE_AUDIT` (run_id, operation, status, details, timestamps)

## 12. Verification

**Module: `src/carelens/verify.py` (60 lines)**

Post-deployment account validation against the local oracle.

| Check | Expected |
|---|---|
| Entity row counts + unique keys | Match TARGETS from dataset.py |
| MEMBER_360_SNAPSHOT count | 500 |
| MEMBER_RISK_FACTORS count | 3,000 |
| Independent risk oracle comparison | All factors match expected_factors.jsonl |
| Document manifest count | 600 |
| Page coverage | 1,100 distinct document-page combinations |
| Chunk token constraints | Zero violations of 500-900 token range |
| Per-plan Search isolation | SEARCH_PREVIEW returns only matching PLAN_ID |

## 13. Testing

`tests/test_workflow.py` **(318 lines, 21 tests)**

| Category | Tests |
|---|---|
| Risk model | Tier boundaries, invalid scores, zero/full score, duplicate observations, inclusive lookback, latest-lab override |
| Adherence | Overlapping fill deduplication, window clipping |
| Chunking | Measured overlap, residual handling, empty page rejection |
| SQL safety | Identifier injection prevention |
| SQL builders | Foundation DDL, curated views, semantic views, agent spec plan scoping |
| Agent responses | Hidden reasoning stripping, error surfacing, text deduplication, format validation, annotation extraction |
| Dataset (integration) | Full volume validation, PDF integrity, SHA-256, condition coverage, risk oracle, reproducibility, seed immutability |
| Deployment | Managed-database guard, document staging safety, app sharing restriction, PUT path handling |
| Governance | Row access policy wiring, risk lineage SOURCE_IDS, warehouse injection |

`tests/test_docs.py` **(38 lines, 2 tests)**

| Test | Validates |
|---|---|
| `test_portable_skill_alias_and_frontmatter` | 12 skills exist with valid YAML frontmatter; .snowflake/cortex/skills mirrors .cortex/skills |
| `test_project_kit_local_links_and_anchors` | All Markdown links and heading anchors resolve across README, AGENTS.md, docs/, .cortex/ |

## 14. Evaluation Cases

### Golden Questions (G01-G08)

| ID | Question | Tools Expected |
|---|---|---|
| G01 | Summarize member P001 | Analyst + Search |
| G02 | Why is P001 high risk? | Analyst (risk factors) |
| G03 | What are P001's recent abnormal labs? | Analyst |
| G04 | What did the latest note say about medication adherence? | Search |
| G05 | Which guideline evidence is relevant? | Search (shared) |
| G06 | How many high-risk members have a follow-up gap? | Analyst |
| G07 | What safety reports mention Drug A? | Search (shared) |
| G08 | What evidence supports this answer? | Both |

### Negative/Boundary Cases (N01-N06)

| ID | Scenario | Expected Behavior |
|---|---|---|
| N01 | Unknown condition | Safe fallback: insufficient evidence |
| N02 | Unknown member | Safe fallback: member not found |
| N03 | Treatment request | Scope refusal |
| N04 | Missing document type | Safe fallback: no matching documents |
| N05 | Missing lab inputs | Reports UNKNOWN state, not low risk |
| N06 | Conflicting evidence (P002) | Reports both sources without silently choosing one |

## 15. Project File Map

| Path | Purpose |
|---|---|
| `AGENTS.md` | Project instructions and implementation rules for AI assistants |
| `README.md` | Project overview and quick-start guide |
| `pyproject.toml` | Python package definition (carelens360 v0.1.0, Python ≥3.9) |
| `config/workflow.json` | Central configuration: database, seed, as-of date, risk thresholds, refresh schedule |
| `src/carelens/cli.py` | CLI entry point: generate, render-sql, check-access, deploy |
| `src/carelens/dataset.py` | Deterministic synthetic data generation from seed corpus |
| `src/carelens/sql.py` | All SQL DDL/DML generation (foundation, curated, governed, semantic) |
| `src/carelens/risk.py` | Independent six-factor risk scoring oracle |
| `src/carelens/chunks.py` | Page-local measured chunking with binary search |
| `src/carelens/agent.py` | Cortex Agent YAML specification generator |
| `src/carelens/response.py` | Agent response parser and format validator |
| `src/carelens/deploy.py` | Six-phase Snowflake deployment with evidence logging |
| `src/carelens/verify.py` | Post-deployment account validation against oracle |
| `snowflake/refresh.py` | Snowpark stored procedure: incremental parse/chunk pipeline |
| `app/streamlit_app.py` | Four-screen Streamlit evidence copilot |
| `app/environment.yml` | Conda environment for Snowflake-hosted Streamlit (channel `snowflake`: streamlit, snowflake-snowpark-python; no python pin, which the warehouse runtime rejects) |
| `tests/test_workflow.py` | 22 test functions (42 test cases across both test files, with parametrization): risk model, chunking, SQL, agents, dataset, governance |
| `tests/test_docs.py` | 2 tests: skill frontmatter, Markdown link integrity |
| `docs/` | 11 contract documents (architecture, scope, risk model, agent contract, governance, evaluation, demo runbook, etc.) |
| `.cortex/skills/` | 12 CoCo skills for developer-assistant workflows |
| `.cortex/memory/` | 6 curated project memory files (context, decisions, status, handoff) |
| `scripts/legacy/` | 5 original draft files (SQL, YAML, PowerShell) preserved unchanged |

## 16. CoCo Skills

| # | Skill | Purpose |
|---|---|---|
| 1 | synthetic-healthcare-data-generator | Generate deterministic full-MVP fixtures with all linked entities |
| 2 | document-evidence-extractor | Build/validate document manifests, AI_PARSE_DOCUMENT, chunks, Search indexing |
| 3 | member-360-builder | Build curated dimensions, facts, and MEMBER_360_SNAPSHOT |
| 4 | risk-factor-explainer | Implement/test six-factor risk scores with tier boundaries and source evidence |
| 5 | citation-validator | Validate claims against structured lineage or document excerpts |
| 6 | clinical-question-evaluator | Run golden questions, structured probes, and boundary tests |
| 7 | healthcare-agent-guardrails | Apply evidence-only agent instructions and safe-fallback behavior |
| 8 | carelens-semantic-agent-builder | Create semantic view, Analyst metrics, Search tool, Agent orchestration |
| 9 | carelens-streamlit-builder | Build/validate all four Streamlit screens |
| 10 | carelens-governance-bootstrap | Implement five-role governance, row filtering, classification, audits |
| 11 | carelens-lifecycle-demo | Capture CoCo lifecycle artifacts and rehearse the demo |
| 12 | carelens-project-memory | Load context, resume work, save decisions and handoffs |

## 17. Configuration Reference

`config/workflow.json`

| Key | Value | Key | Value |
|---|---|---|---|
| database | `CARELENS360_MVP` | adherence_threshold | 0.8 |
| connection_name | `wtojjon-eyc21064` | chronic_condition_minimum | 2 |
| seed | 42 | adverse_event_window_days | 90 |
| as_of_date | `2026-09-30` | high_risk_tiers | [High, Critical] |
| rule_version | `synthetic-demo-v1` | parse_batch_size | 10 |
| lab_window_days | 180 | search_target_lag | 1 hour |
| ed_window_days | 90 | refresh_schedule | 60 MINUTE |
| adherence_window_days | 183 | refresh_enabled | false |